

Hour Booking System - Complete Setup Guide

Hour Booking System - End-to-End Guide

Staff hours booking and project budget tracker: staff/admin roles, WBS projects, cost calculation. Stack: MySQL, Java (Spring Boot) backend, HTML/CSS/JavaScript frontend.

Step 0: Create the Spring Boot Project

Before writing any code, generate the project skeleton so the required JDBC driver and JPA starter are on the classpath from the start.

1. Go to start.spring.io (Spring Initializr).
2. Set:
 - **Project:** Maven
 - **Language:** Java
 - **Spring Boot version:** latest stable 3.x
 - **Group:** com.example
 - **Artifact / Name:** demo
 - **Packaging:** Jar
 - **Java:** 17 (or 21)
3. Add these **Dependencies**:
 - Spring Web
 - Spring Data JPA (pulls in Hibernate, which talks to MySQL through JDBC)
 - MySQL Driver (this is the actual JDBC driver - mysql-connector-j)
4. Click **Generate**, unzip the download, and open it in your IDE (IntelliJ / Eclipse / VS Code).

If you already created the project without selecting these, just add the dependencies to pom.xml manually (Step 1 below).

pom.xml - confirm the JDBC driver is present

This is the part that actually makes JDBC work. spring-boot-starter-data-jpa gives you Hibernate/JPA, but Hibernate still needs a **JDBC driver** underneath it to talk to MySQL - that driver is mysql-connector-j. Make sure your pom.xml has all three:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- JDBC driver for MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
```

```
<scope>runtime</scope>
</dependency>
</dependencies>
```

application.properties - the JDBC connection itself

This is where JDBC is configured directly: the `spring.datasource.url` line is a standard **JDBC URL** (`jdbc:mysql://...`), and `driver-class-name` points at the JDBC driver class. Create `src/main/resources/application.properties`:

```
# JDBC connection to MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/hour_booking_db?
useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=your_mysql_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate (uses the JDBC connection above)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

```
server.port=8080
```

How the pieces connect: Controller → Repository (Spring Data JPA) → Hibernate → JDBC driver (`mysql-connector-j`) → MySQL, using the JDBC URL above. So yes - JDBC is used, just under the hood of JPA rather than through raw `java.sql.Connection` code. If you'd rather see it explicitly, you can swap any repository method for plain `JdbcTemplate` calls, but for this project JPA is the simpler, recommended path.

Step 1: Database Setup (MySQL Workbench)

Open MySQL Workbench, run this complete script to set up your database, tables, relationships, and test data:

```
CREATE DATABASE IF NOT EXISTS hour_booking_db;
USE hour_booking_db;

CREATE TABLE users (
  username VARCHAR(50) PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL
);

CREATE TABLE staff_table (
  member_id VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  base_rate DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (member_id) REFERENCES users(username) ON DELETE
CASCADE
);

CREATE TABLE project_table (
  wbs_code VARCHAR(50) PRIMARY KEY,
  project_name VARCHAR(100) NOT NULL,
  expiry_date DATE NOT NULL,
  amount_allocated DECIMAL(15,2) NOT NULL,
  amount_available DECIMAL(15,2) NOT NULL,
  amount_booked DECIMAL(15,2) DEFAULT 0.00
);
```

```

CREATE TABLE project_member_mapping_table (
    mapping_id INT AUTO_INCREMENT PRIMARY KEY,
    wbs_code VARCHAR(50),
    member_id VARCHAR(50),
    allocation_percentage DECIMAL(5,2),
    FOREIGN KEY (wbs_code) REFERENCES project_table(wbs_code),
    FOREIGN KEY (member_id) REFERENCES staff_table(member_id)
);

CREATE TABLE booking_table (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    wbs_code VARCHAR(50),
    member_id VARCHAR(50),
    hours_booked DECIMAL(6,2),
    month INT,
    year INT,
    calculated_cost DECIMAL(15,2),
    status VARCHAR(30) DEFAULT 'Pending Review',
    FOREIGN KEY (wbs_code) REFERENCES project_table(wbs_code),
    FOREIGN KEY (member_id) REFERENCES staff_table(member_id)
);

-- Test Records
INSERT INTO users VALUES ('staff01', 'John Doe', 'staff123', 'staff');
INSERT INTO staff_table VALUES ('staff01', 'John Doe', 500.00);
INSERT INTO project_table VALUES ('WBS101', '2D Active Missile Detection Radar', '2026-12-31', 100000.00, 100000.00, 0.00);
INSERT INTO project_member_mapping_table (wbs_code, member_id, allocation_percentage) VALUES ('WBS101', 'staff01', 25.00);

```

Step 2: Spring Boot Backend Code

1. Entities (src/main/java/com/example/demo/entity/)

Create the entity classes (User.java, Staff.java, Project.java, ProjectMemberMapping.java, Booking.java) mapped to your database tables using JPA.

2. Repositories

(src/main/java/com/example/demo/repository/)

Create standard JpaRepository interfaces for UserRepository, StaffRepository, ProjectRepository, BookingRepository, and ProjectMemberMappingRepository:

```

package com.example.demo.repository;
import com.example.demo.entity.ProjectMemberMapping;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface ProjectMemberMappingRepository extends
JpaRepository<ProjectMemberMapping, Long> {
    List<ProjectMemberMapping> findByMemberId(String memberId);
    boolean existsByWbsCodeAndMemberId(String wbsCode, String
memberId);
}

```

3. Main Controller

(src/main/java/com/example/demo/controller/AppController.java)

This handles all business logic, enforcing Expiry Locks, Staff Mapping Validation, and Financial Calculations:

```
package com.example.demo.controller;

import com.example.demo.entity.*;
import com.example.demo.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class AppController {

    @Autowired private UserRepository userRepository;
    @Autowired private StaffRepository staffRepository;
    @Autowired private ProjectRepository projectRepository;
    @Autowired private BookingRepository bookingRepository;
    @Autowired private ProjectMemberMappingRepository
mappingRepository;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody User loginUser) {
        User user =
userRepository.findById(loginUser.getUsername()).orElse(null);
        if (user != null &&
user.getPassword().equals(loginUser.getPassword())) {
            return ResponseEntity.ok(user);
        }
        return ResponseEntity.status(401).body("Invalid
credentials");
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody User newUser) {
        if (userRepository.existsById(newUser.getUsername())) {
            return ResponseEntity.badRequest().body("Username
already exists!");
        }
        userRepository.save(newUser);
        if ("staff".equalsIgnoreCase(newUser.getRole())) {
            Staff staff = new Staff();
            staff.setMemberId(newUser.getUsername());
            staff.setName(newUser.getFullName());
            staff.setBaseRate(500.00);
            staffRepository.save(staff);
        }
        return ResponseEntity.ok("Registration successful");
    }

    @GetMapping("/mapped-projects/{memberId}")
    public List<ProjectMemberMapping>
getMappedProjects(@PathVariable String memberId) {
        return mappingRepository.findById(memberId);
    }

    @PostMapping("/book-hours")
    public ResponseEntity<?> bookHours(@RequestBody Booking booking)
{
        Project project =
projectRepository.findById(booking.getWbsCode()).orElse(null);
        if (project == null) return
ResponseEntity.badRequest().body("Invalid WBS Code");

        // Expiry Lock Check
```

```

        if (LocalDate.now().isAfter(project.getExpiryDate())) {
            return ResponseEntity.badRequest().body("Project is
expired! Booking blocked.");
        }

        // Staff Mapping Validation
        boolean isMapped =
mappingRepository.existsByWbsCodeAndMemberId(booking.getWbsCode(),
booking.getMemberId());
        if (!isMapped) {
            return ResponseEntity.badRequest().body("Staff is not
mapped to this project.");
        }

        Staff staff =
staffRepository.findById(booking.getMemberId()).orElse(null);
        double cost = booking.getHoursBooked() *
staff.getBaseRate(); // Financial Calculation
        booking.setCalculatedCost(cost);
        booking.setStatus("Pending Review");

        project.setAmountBooked(project.getAmountBooked() + cost);
        project.setAmountAvailable(project.getAmountAllocated() -
project.getAmountBooked());

        projectRepository.save(project);
        bookingRepository.save(booking);

        return ResponseEntity.ok("Hours booked successfully and
marked as Pending Review!");
    }

    @GetMapping("/bookings")
    public List<Booking> getAllBookings() {
        return bookingRepository.findAll();
    }

    @PostMapping("/projects")
    public ResponseEntity<?> createProject(@RequestBody Project
project) {
        project.setAmountAvailable(project.getAmountAllocated() -
(project.getAmountBooked() != null ? project.getAmountBooked() :
0.0));
        projectRepository.save(project);
        return ResponseEntity.ok("Project created successfully");
    }

    @PostMapping("/map-member")
    public ResponseEntity<?> mapMember(@RequestBody
ProjectMemberMapping mapping) {
        mappingRepository.save(mapping);
        return ResponseEntity.ok("Member mapped successfully");
    }
}

```

Step 3: Front-End Files (src/main/resources/static/)

Place these HTML, CSS, and JS files inside your Spring Boot static resources folder:

1. index.html (Sign In)

```

<!DOCTYPE html>
<html lang="en">

```

```

<head><title>Sign In</title><link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h2>Sign In</h2>
    <form id="loginForm">
      <label>Username</label><input type="text" id="username"
required>
      <label>Password</label><input type="password"
id="password" required>
      <button type="submit">Sign In</button>
    </form>
    <p class="switch-text">Don't have an account? <a
href="register.html">Sign Up</a></p>
  </div>
  <script src="script.js"></script>
</body>
</html>

```

2. register.html (Sign Up)

```

<!DOCTYPE html>
<html lang="en">
<head><title>Sign Up</title><link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h2>Create Account</h2>
    <form id="registerForm">
      <label>Full Name</label><input type="text" id="fullName"
required>
      <label>Username</label><input type="text" id="username"
required>
      <label>Password</label><input type="password"
id="password" required>
      <label>Role</label>
      <select id="role"><option value="staff">Staff</option>
<option value="admin">Admin</option></select>
      <button type="submit">Register</button>
    </form>
    <p class="switch-text">Already have an account? <a
href="index.html">Sign In</a></p>
  </div>
  <script src="script.js"></script>
</body>
</html>

```

3. staff-dashboard.html (Staff View)

```

<!DOCTYPE html>
<html lang="en">
<head><title>Staff Dashboard</title><link rel="stylesheet"
href="style.css"></head>
<body>
  <div class="card" style="width: 500px;">
    <h2>Staff Hour Booking</h2>
    <form id="bookingForm">
      <label>Staff ID</label><input type="text" id="staffId"
value="staff01" required>
      <label>WBS Code</label><input type="text" id="wbsCode"
required placeholder="e.g. WBS101">
      <label>Month (1-12)</label><input type="number"
id="month" required>
      <label>Year</label><input type="number" id="year"
value="2026" required>
      <label>Hours Booked</label><input type="number"
step="0.5" id="hours" required>
    </form>
  </div>

```

```

        <button type="submit">Submit Hours (Pending Review)
    </button>
    </form>
  </div>
  <script src="script.js"></script>
</body>
</html>

```

4. admin-dashboard.html (Admin View)

```

<!DOCTYPE html>
<html lang="en">
<head><title>Admin Dashboard</title><link rel="stylesheet"
href="style.css"></head>
<body>
  <div class="card" style="width: 700px;">
    <h2>Admin Management Panel</h2>
    <h3>Create Project</h3>
    <form id="projectForm">
      <input type="text" id="projWbs" placeholder="WBS Code"
required>
      <input type="text" id="projName" placeholder="Project
Name" required>
      <input type="date" id="projExpiry" required>
      <input type="number" id="projAllocated"
placeholder="Amount Allocated" required>
      <button type="submit">Save Project</button>
    </form>
  </div>
  <script src="script.js"></script>
</body>
</html>

```

5. style.css

```

body { font-family: Arial, sans-serif; background: #f4f7f6; padding:
30px; }
.card { background: white; padding: 25px; border-radius: 8px; width:
380px; margin: 20px auto; box-shadow: 0 4px 10px rgba(0,0,0,0.1); }
h2, h3 { text-align: center; color: #0056b3; }
label { display: block; margin-top: 10px; font-weight: bold; font-
size: 14px; }
input, select { width: 100%; padding: 10px; margin-top: 5px; box-
sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
button { width: 100%; background: #007bff; color: white; border:
none; padding: 12px; margin-top: 15px; border-radius: 4px; cursor:
pointer; font-weight: bold; }
button:hover { background: #004085; }
.switch-text { text-align: center; margin-top: 15px; font-size:
14px; }

```

6. script.js

```

document.getElementById("loginForm")?.addEventListener("submit",
async (e) => {
  e.preventDefault();
  const res = await fetch("http://localhost:8080/api/login", {
    method: "POST", headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({ username:
document.getElementById("username").value, password:
document.getElementById("password").value })
  });
  if (res.ok) {
    const user = await res.json();
    alert("Login successful! Role: " + user.role);
    window.location.href = user.role === 'admin' ? 'admin-

```

```

dashboard.html' : 'staff-dashboard.html';
    } else { alert("Invalid credentials."); }
  });

  document.getElementById("registerForm").addEventListener("submit",
async (e) => {
    e.preventDefault();
    const res = await fetch("http://localhost:8080/api/register", {
      method: "POST", headers: { "Content-Type":
"application/json" },
      body: JSON.stringify({
        fullName: document.getElementById("fullName").value,
        username: document.getElementById("username").value,
        password: document.getElementById("password").value,
        role: document.getElementById("role").value
      })
    });
    if (res.ok) { alert("Registration successful!");
window.location.href = "index.html"; }
    else { alert("Registration failed."); }
  });

  document.getElementById("bookingForm").addEventListener("submit",
async (e) => {
    e.preventDefault();
    const data = {
      memberId: document.getElementById("staffId").value,
      wbsCode: document.getElementById("wbsCode").value,
      month: parseInt(document.getElementById("month").value),
      year: parseInt(document.getElementById("year").value),
      hoursBooked:
parseFloat(document.getElementById("hours").value)
    };
    const res = await fetch("http://localhost:8080/api/book-hours",
{
      method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(data)
    });
    alert(await res.text());
  });

```

Step 4: How to Run the Application

1. **Database:** Execute the SQL script from Step 1 in MySQL Workbench.
2. **Backend:** Open the Spring Boot project (created in Step 0) in your IDE, fill in your real MySQL username/password in application.properties (Step 0), and run the main @SpringBootApplication class.
3. **Frontend:** Open your browser and navigate to <http://localhost:8080/index.html> to test sign-in, registration, and role-based views.

JDBC checklist

- ☒ mysql-connector-j dependency in pom.xml (the JDBC driver itself)
- ☒ spring.datasource.url=jdbc:mysql://... in application.properties (the JDBC URL)
- ☒ spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver set explicitly
- ☒ Hibernate/JPA sits on top of this JDBC connection - every repository call (save, findById, findAll, etc.) ultimately goes through it